

CGU-NLP-Classification

Environment types, Running environment, Python version: Colab

AI三 B1228005 胡樂麒

1. TODO6

1. 你使用的 Claude Code 版本（執行 'claude --version' 取得）

```
C:\Users\User>claude --version
2.1.100 (Claude Code)
```

2. 你的操作過程：你給予 Claude Code 的提示詞、Claude Code 的回應摘要（複製貼上或截圖）、以及你進行了幾輪對話來完成

我：

請幫我寫一個 Python 腳本 plot_confusion_matrix.py，做以下事情：

1. 讀取當前目錄下的 test_predictions.npz 檔案
2. 取出真實標籤（true labels）和預測標籤（predicted labels）
3. 使用 scikit-learn 計算 confusion matrix
4. 用 matplotlib 繪製 confusion matrix，需包含每格的數值標籤
5. 類別名稱為 ["World", "Sports", "Business", "Sci/Tech"]
6. 儲存圖片為 confusion_matrix.png

claude code:

```
• Write(plot_confusion_matrix.py)
Wrote 35 lines to plot_confusion_matrix.py
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

data = np.load("test_predictions.npz")
true_labels = data["true_labels"]
pred_labels = data["pred_labels"]

class_names = ["World", "Sports", "Business", "Sci/Tech"]
... +25 lines (ctrl+ to expand)
腳本已建立於 C:\Users\User\Downloads\plot_confusion_matrix.py。
使用方式：
python plot_confusion_matrix.py
腳本流程說明：
1. 讀取資料 - np.load("test_predictions.npz") 讀取目錄下的 test_predictions.npz，取出 true_labels 和 pred_labels 兩個陣列。
2. 計算混淆矩陣 - sklearn.metrics.confusion_matrix 計算 true 與 pred。
3. 繪圖 - plt.imshow 顯示矩陣，並在每個格子標上數值；黃格子顏色較深，文字用白色，較淺的用黑色，確保可讀性。
4. 儲存 - 輸出為 confusion_matrix.png (300 DPI)。
注意：test_predictions.npz 中的陣列 key 可能是 true_labels 和 pred_labels，若你的檔案使用不同的 key 名稱（例如 y_true / y_pred），請對應修改第 5-6 行。
請執行 python plot_confusion_matrix.py
• Bash(cd "C:\Users\User\Downloads" && python plot_confusion_matrix.py)
! Error: Exit code 1
Traceback (most recent call last):
  File "C:\Users\User\Downloads\plot_confusion_matrix.py", line 1, in <module>
    import numpy as np
ModuleNotFoundError: No module named 'numpy'
缺少必要套件，先安裝再執行：
```

我：

請執行以下指令確認 python 版本和路徑，然後用對應的 pip 安裝套件：

```
python --version
```

```
where python
```

```
python -m pip install numpy matplotlib scikit-learn
```

然後再執行 `python plot_confusion_matrix.py`

claude code:

```
• Bash(cd "C:\Users\User\Downloads" && python --version && where python)
  Python 3.12.18
  C:\Users\User\AppData\Local\Microsoft\WindowsApps\python.exe

• Bash(cd "C:\Users\User\Downloads" && python -m pip install numpy matplotlib scikit-learn)
  Defaulting to user installation because normal site-packages is not writeable
  Collecting numpy
    Downloading numpy-2.0.0-cp312-cp312-win_amd64.whl.metadata (6.6 kB)
    (lines 26)

  操作安裝成功，現在執行腳本：

• Bash(cd "C:\Users\User\Downloads" && python plot_confusion_matrix.py)
  Traceback (most recent call last):
    File "C:\Users\User\Downloads\plot_confusion_matrix.py", line 6, in <module>
      true_labels = data["true_labels"]
  KeyError: 'true_labels'

  File "C:\Users\User\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12.qbz0z6frdp8\LocalCache\local-packages\Python312\site-packages\numpy\_lib\_array\_nditer.py", line 240, in _nditer._
    raise KeyError(f'{key} is not a file in the archive') from None
  KeyError: 'true_labels is not a file in the archive'

  腳本可以執行，但 test_predictions.npz 裡的陣列 key 名稱不是 true_labels，而是和實際的 key：
  Interpreter: What should Claude do instead?
```

我：

請修改 `plot_confusion_matrix.py`，將第 6、7 行的 key 名稱改為：

```
true_labels = data["labels"]
```

```
pred_labels = data["preds"]
```

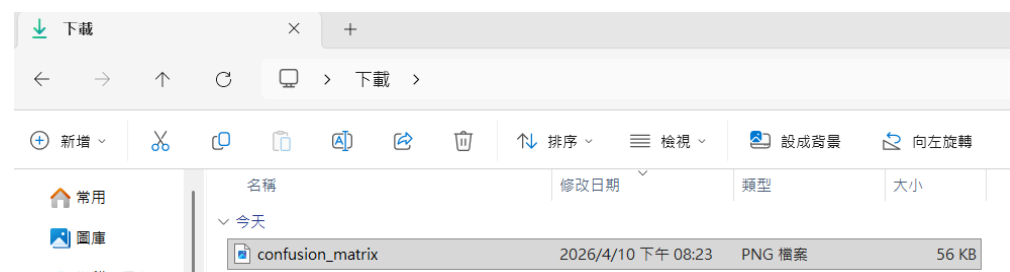
然後重新執行腳本

claude code:

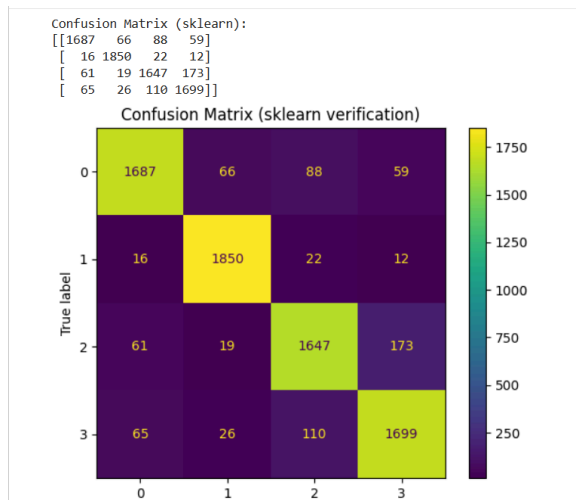
```
• Update(plot_confusion_matrix.py)
  Added 2 lines, removed 2 lines
  3 from sklearn.metrics import confusion_matrix
  4
  5 data = np.load("test_predictions.npz")
  6 -true_labels = data["true_labels"]
  7 -pred_labels = data["pred_labels"]
  6 +true_labels = data["labels"]
  7 +pred_labels = data["preds"]
  8
  9 class_names = ["World", "Sports", "Business", "Sci/Tech"]
  10

• Bash(cd "C:\Users\User\Downloads" && python plot_confusion_matrix.py)
  Saved confusion_matrix.png

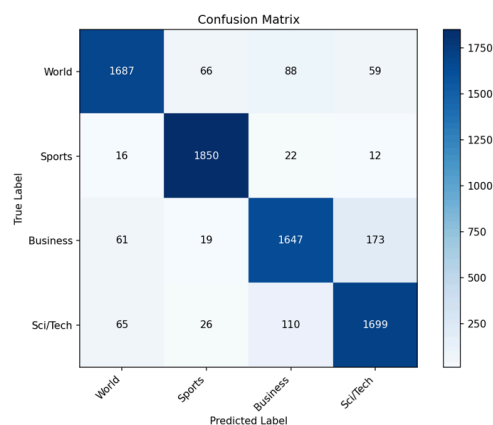
  執行成功，confusion_matrix.png 已儲存至 C:\Users\User\Downloads\。
```



3. 你如何驗證程式碼的正確性？ 我使用 sklearn 的 ‘ConfusionMatrixDisplay’ 寫一段程式碼(程式碼在colab中最後一格cell)。



4. 於報告中附上最終的 ‘confusion_matrix.png’ 圖片



Model performance :0.9057

```
100%|██████████| 1500/1500 [00:26<00:00, 56.42it/s]
Start evaluating the model on the training set.
OK| | 0/1500 [00:00<?, ?it/s]/usr/local/lib/python3.12/dist-packages/torch/nn/modules/transformer.py:531: UserWarning: The PyTorch API of nested tensors is in prototype stage and will change in
output = torch.nested_tensor_from_mask(
100%|██████████| 1500/1500 [00:10<00:00, 141.76it/s]
Start evaluating the model on the validation set.
100%|██████████| 188/188 [00:02<00:00, 65.03it/s]
Epoch [1/5] Loss: 0.5784 | Train Acc: 0.8801 | Validation Acc: 0.8709 | Time: 26.6s
100%|██████████| 1500/1500 [00:31<00:00, 47.02it/s]
Start evaluating the model on the training set.
100%|██████████| 1500/1500 [00:16<00:00, 93.27it/s]
Start evaluating the model on the validation set.
100%|██████████| 188/188 [00:02<00:00, 64.08it/s]
Epoch [2/5] Loss: 0.3652 | Train Acc: 0.9040 | Validation Acc: 0.8916 | Time: 31.9s
100%|██████████| 1500/1500 [00:26<00:00, 57.61it/s]
Start evaluating the model on the training set.
100%|██████████| 1500/1500 [00:10<00:00, 141.38it/s]
Start evaluating the model on the validation set.
100%|██████████| 188/188 [00:02<00:00, 82.47it/s]
Epoch [3/5] Loss: 0.3130 | Train Acc: 0.9160 | Validation Acc: 0.8956 | Time: 26.0s
100%|██████████| 1500/1500 [00:26<00:00, 56.62it/s]
Start evaluating the model on the training set.
100%|██████████| 1500/1500 [00:10<00:00, 141.43it/s]
Start evaluating the model on the validation set.
100%|██████████| 188/188 [00:02<00:00, 82.64it/s]
Epoch [4/5] Loss: 0.2824 | Train Acc: 0.9246 | Validation Acc: 0.9020 | Time: 26.5s
100%|██████████| 1500/1500 [00:26<00:00, 56.00it/s]
Start evaluating the model on the training set.
100%|██████████| 1500/1500 [00:10<00:00, 138.91it/s]
Start evaluating the model on the validation set.
100%|██████████| 188/188 [00:02<00:00, 83.19it/s]
Epoch [5/5] Loss: 0.2592 | Train Acc: 0.9323 | Validation Acc: 0.9050 | Time: 26.8s
=====
Training complete!
100%|██████████| 60/60 [00:00<00:00, 84.09it/s]Final Test Accuracy: 0.9057
```

請說明你的資料前處理流程

讀取原始資料檔 train.csv 與 test.csv。資料中包含 Class Index、Title 與 Description 三個主要欄位。其中 Class Index 為原始類別標籤，範圍為 1 到 4。為了符合 PyTorch 分類模型的格式，把標籤轉換為從 0 開始，Class Index - 1 後存成新欄位 new_label。

在文字內容處理方面，我定義 title_col = "Title" 與 desc_col = "Description"，並依照 use_agnews_title 參數決定輸入文字的組成方式。當 use_agnews_title=True 時，會將新聞標題與新聞描述以空格串接，把新聞標題與新聞描述一起輸入，使模型能同時利用標題與內文資訊；若設為 False，則只使用 Description 欄位作為輸入文字。

再用 train_test_split() 將原始 train.csv 再切分成訓練集與驗證集，訓練比例設為 train_size =0.8，因此訓練集與驗證集的比例為 80% 與 20%。將切分後資料的文字內容與對應標籤分別轉成 Python list，後續進行 tokenizer 與 Dataset 使用。

文字完成前處理後交給 tokenizer 轉換成模型的編碼格式，input_ids 與 attention_mask。再將編碼後的文字特徵與標籤封裝成 PyTorch Dataset。在 __getitem__() 中，會將每一筆資料轉成 tensor，並回傳包含 input_ids、attention_mask 與 labels 的字典格式，供 DataLoader 訓練、驗證與測試時使用。

請說明你使用的模型以及其超參數

(參數改動的部份我有問ChatGPT推薦要怎麼調整參數)

參數中我調整了，EMBED_DIM(embedding 的維度),FFN_DIM(前饋神經網路維度)提升這兩個參數可以讓模型在向量空間中表示更多語意資訊，使模型更容易區分不同類型的文本，NUM_LAYERS,(Transformer Encoder 的層數)讓模型能夠學習更深層的語意特徵。NUM_HEADS(multi-head-self-attention)模型同時從不同角度學習詞與詞之間的關聯，增加 heads 可以提升模型對語言的理解能力。DROPOUT 防止過擬合, LEARNING_RATE 將數值調小，讓模型在訓練過程中更新更穩定，避免過大的參數變動導致訓練震盪。NUM_EPOCHS 訓練的回數。

Transformer Encoder-based 模型 BERT tokenizer 進行文字編碼進行文本分類			
參數	預設	第一版	第二版
SEED	42	42	42
BATCH_SIZE	64	64	64
TEST_BATCH_SIZE	128	128	128

MAX_SEQ_LEN	128	128	128
EMBED_DIM	128	128	256
NUM_HEADS	4	4	8
NUM_LAYERS	2	3	2
FFN_DIM	256	256	512
DROPOUT	0.1	0.1	0.2
LEARNING_RATE	1e-3	5e-4	5e-4
NUM_EPOCHS	5	6	6
NUM_CLASSES	4	4	4

請製作模型 test set 分數表格，並說明你如何改進你的模型

初始設定作為 baseline，模型已能達到約 0.905 的 test accuracy。所以在第二版實驗中，我透過增加模型深度與調整學習率，然而測試準確率出現略為下降的趨勢，我認為模型可能出現輕微過擬合的現象。在第三版的準確率則介於兩者之間。相較於第二版，第三版將 embedding dimension 由 128 提升至 256，attention heads 由 4 提升至 8，feedforward dimension 由 256 提升至 512，以增強模型對文本特徵的表示能力；同時，將 dropout 由 0.1 提高至 0.2，降低模型容量增加後的過擬合風險，並將 Transformer Encoder 層數由 3 降回 2，避免模型過深造成訓練集表現過強而泛化能力下降。

從混淆矩陣來看則可以發現，其實每一個版本對不同的類別都有不一樣的表現。例如，初版是類別1有1850的正確數，第二版雖然類別1略少於初版，但是類別0則是第二版正確率較高，第三版則是第1跟第3類別正確率特別高，但是0和2則較低。

Model performance		
初版	第二版	第三版
0.9057	0.9029	0.9051

第二版

Start evaluating the model on the validation set.

100% ██████████ 188/188 [00:02<00:00, 77.26it/s]

Epoch [6/6] Loss: 0.2393 | Train Acc: 0.9394 | Validation Acc: 0.9048 | Time: 36.9s

Training complete!

100% ██████████ 60/60 [00:00<00:00, 77.06it/s] Final Test Accuracy: 0.9029

Confusion Matrix (sklearn):

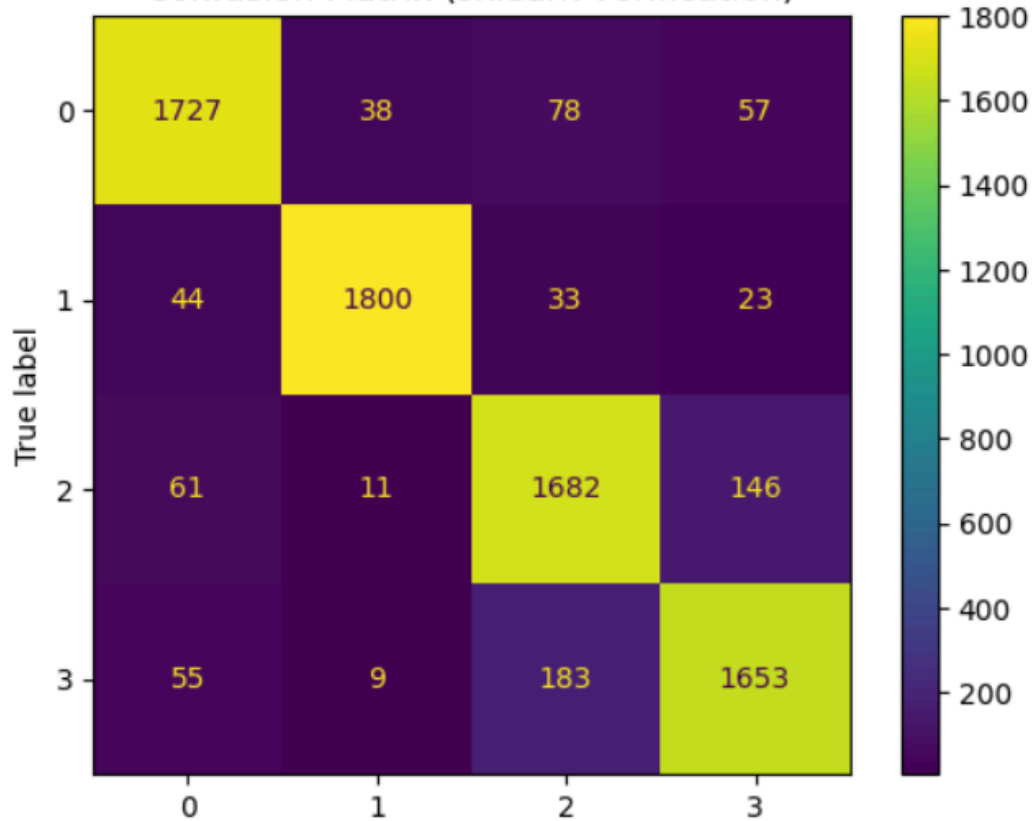
```
[[1727  38  78  57]
```

```
 [ 44 1800  33  23]
```

```
 [ 61  11 1682 146]
```

```
 [ 55   9  183 1653]]
```

Confusion Matrix (sklearn verification)



第三版

Start evaluating the model on the validation set.

100% |██████████████████| 188/188 [00:02<00:00, 74.05it/s]

Epoch [6/6] Loss: 0.2607 | Train Acc: 0.9297 | Validation Acc: 0.9090 | Time: 54.6s

=====

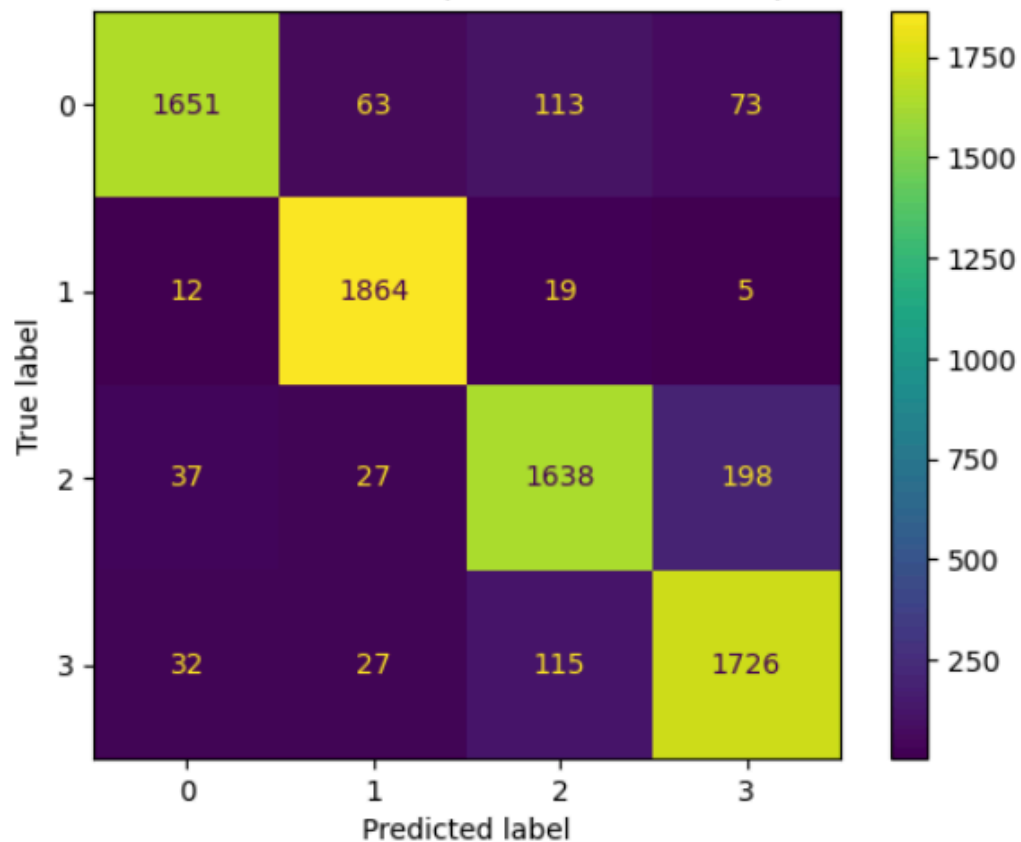
Training complete!

100% |██████████████████| 60/60 [00:01<00:00, 59.44it/s]Final Test Accuracy: 0.9051

Confusion Matrix (sklearn):

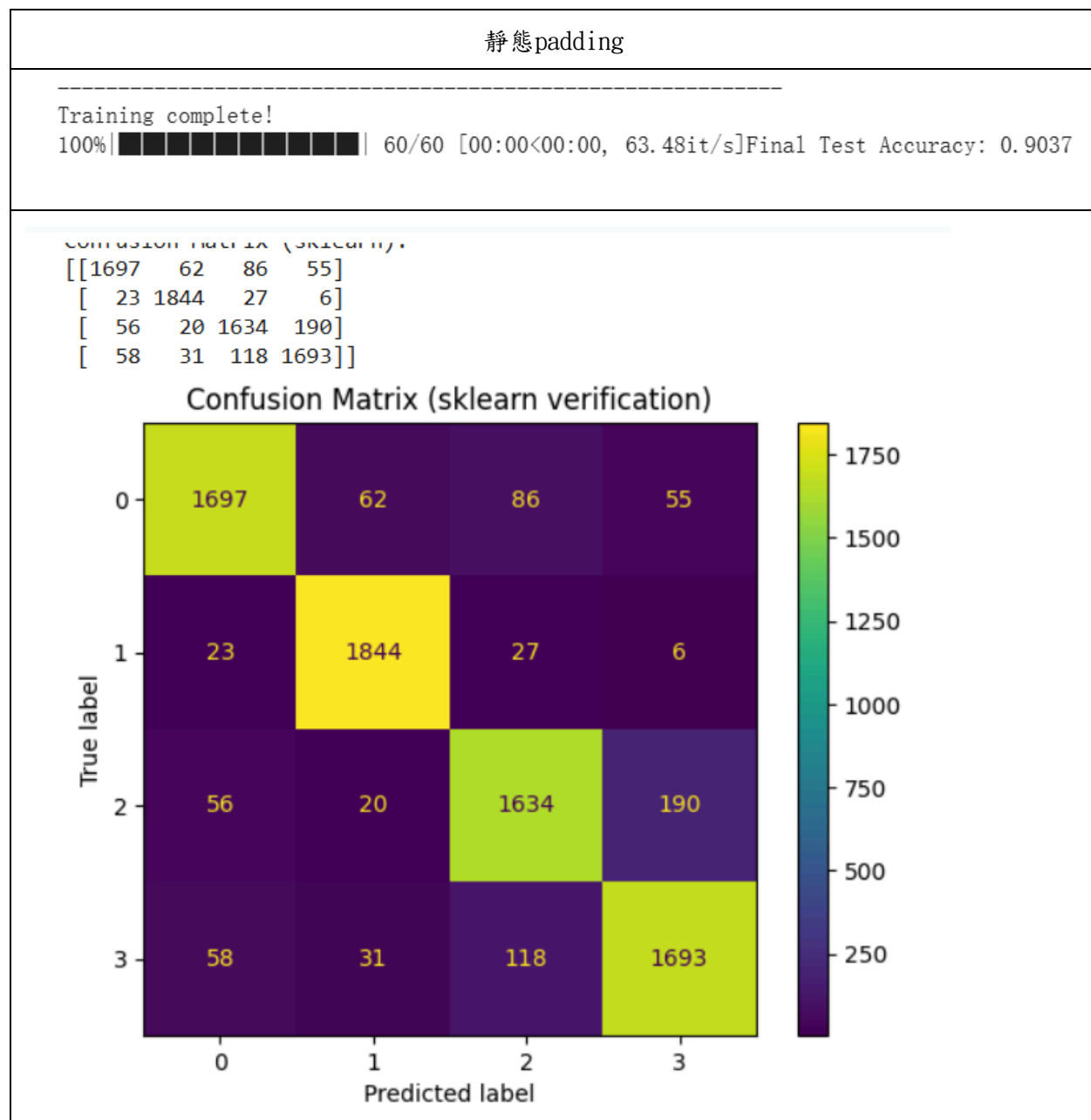
```
[[1651  63  113  73]
 [  12 1864  19   5]
 [  37  27 1638 198]
 [  32  27  115 1726]]
```

Confusion Matrix (sklearn verification)



請比較有無進行動態 padding 的模型 test set 分數差異

因為在準確率上表現最好的是我初版的參數，因此我使用初版的模型來進行有無進行動態 padding 的比較(初版的動態padding結果請參考todo6的區塊)，靜態padding的結果略輸於動態padding但是結果相差不大。我覺得是因為靜態padding他會將所有句子補齊至相同長度，導致模型在計算過程中需處理大量無效的空白 token 所以對結果造成了些微影響。



Anything that can strengthen your report.

請詳細說明‘ TransformerClassifier ’ 的 Step 6 在做什麼畫更多的結果圖

`attention_mask` 是一個二維矩陣，用來標記輸入序列中哪些位置為真實 token（值為 1），哪些為 padding（值為 0）。透過 `unsqueeze (-1)` 在最後新增一個維度，使其形狀能與 `encoder_output` 對齊，並轉為浮點數以利後續運算。接著，將 `encoder_output` 與 `mask_expanded` 相乘，使 padding 對應位置的向量被歸零，只保留真實的 token。再沿著 token 進行加總，得到所有有效 token 向量的總和。並將 `attention_mask` 加總來計算每個句子中真實 token 的數量，並使用 `clamp (min=1e-9)` 避免在極端情況下出現除以零的錯誤。最後，將向量總和除以真實 token 的數量，得到該句子的平均向量。這個過程稱為 `masked mean pooling`，目的在於將 Transformer 輸出的 token-level 表示轉換為固定長度的 sentence-level，供後續分類器使用，同時避免 padding 對結果造成影響。總而言之，他對所有真實 token 的向量取平均，並忽略 padding 的部分，以取得整句的語意。（上面的解釋是我問ChatGPT並用自己的話來闡述）

Claude code產生混淆矩陣的程式碼

```
請顯示目前 plot_confusion_matrix.py 的完整內容

Read 1 file (ctrl+o to expand)

以下是目前 plot_confusion_matrix.py 的完整內容：

import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

data = np.load("test_predictions.npz")
true_labels = data["labels"]
pred_labels = data["preds"]

class_names = ["World", "Sports", "Business", "Sci/Tech"]

cm = confusion_matrix(true_labels, pred_labels)

fig, ax = plt.subplots(figsize=(8, 6))
im = ax.imshow(cm, interpolation="nearest", cmap=plt.cm.Blues)
plt.colorbar(im, ax=ax)

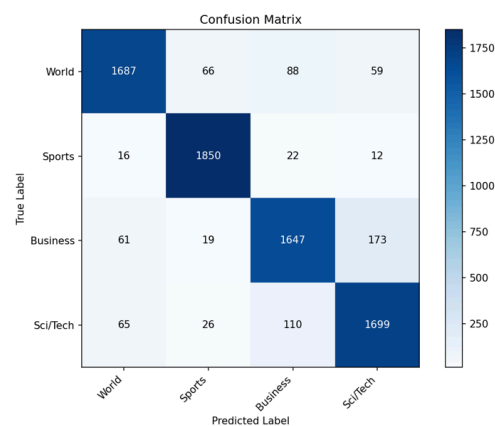
ax.set_xticks(range(len(class_names)))
ax.set_yticks(range(len(class_names)))
ax.set_xticklabels(class_names, rotation=45, ha="right")
ax.set_yticklabels(class_names)

thresh = cm.max() / 2
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        ax.text(j, i, cm[i, j],
                ha="center", va="center",
                color="white" if cm[i, j] > thresh else "black")

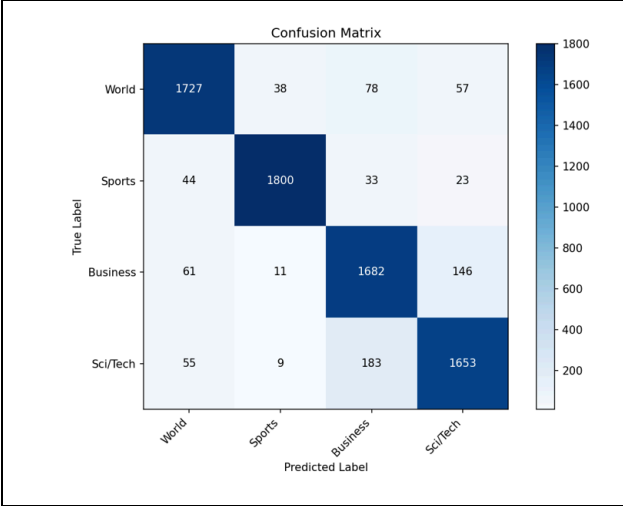
ax.set_xlabel("Predicted Label")
ax.set_ylabel("True Label")
ax.set_title("Confusion Matrix")

plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=150)
print("Saved confusion_matrix.png")
```

初版混淆矩陣圖片(Claude code生成)



第二版混淆矩陣圖片(Claude code生成)



第三版混淆矩陣圖片(Claude code生成)

